

THE UNIVERSITY OF ZAMBIA

SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT PROPOSAL (2026)

Project Title:

ConferenceCheck Mobile: Analytics, Meals and Notifications

Student Name:

Ndashi Bwalya Chisanga

Computer Number:

2021470105

Supervisor:

Mr. Mofya Phiri

Department:

Computing and Informatics

Date of Submission:

28/03/26 (version 4, resubmitted 10/10/26)

Table of Contents

Abstract.....	4
1. Introduction.....	4
1.1 Problem Statement.....	5
1.2 Project Aim.....	6
1.3 Objectives.....	6
1.4 Scope of the Project	7
2. Literature Review	9
3. Proposed Methodology	11
3.1 System Architecture.....	11
3.2 Meal Voucher Redemption Protocol.....	12
3.3 Analytics Dashboard.....	13
3.4 Session Attendance and Capacity Management	13
3.5 Push Notification System.....	13
3.6 Reporting and Data Export.....	14
3.7 Tools and Technologies.....	14
3.8 Ethical and Legal Considerations.....	14
4. Expected Outcomes.....	15
5. Evaluation Methodology	15
6. Project Timeline	17
7. Risk Analysis	17
8. Limitations.....	18
9. AI-Assisted Development Statement	19
9.1 AI Tools to be Used	19
9.2 Intended Tasks and Scope of Assistance.....	19

9.3 Validation of AI-Generated Code.....	19
9.4 Documentation of AI Usage	19
10. Remaining Work.....	21
11. References	22

Abstract

On-site conference operations such as meal service, session seating, and attendee communication are still commonly managed with paper registers, printed vouchers, and ad-hoc announcements. These manual processes scale poorly, provide organisers with no live operational picture, and are vulnerable to fraud such as duplicate meal collection. This project proposes the design and implementation of ConferenceCheck Mobile: Analytics, Meals and Notifications, component 4.3b of a two-part conference management companion application. Component 4.3a, developed separately, handles the core attendee check-in and offline synchronisation engine. This project builds on top of that check-in data to deliver four equally central capabilities: a real-time analytics dashboard that turns raw check-in and attendance figures into actionable operational insight; a QR-code based meal voucher system in which single-use redemption is enforced server-side through database-level uniqueness constraints and immutable redemption records; session attendance tracking with configurable capacity thresholds and overcrowding alerts pushed to organisers; and a push notification system built on Firebase Cloud Messaging with per-recipient delivery auditing. The system also includes CSV reporting and data export for post-event use. Check-in data is consumed through a defined REST API contract shared with component 4.3a, and this component can be developed, tested, and graded independently using seeded check-in fixtures. The system comprises a Flutter mobile application backed by a Laravel REST API and a PostgreSQL datastore. Evaluation is conducted against explicit, measurable acceptance criteria, including rejection of concurrent duplicate redemptions, median scan-validation latency, notification delivery rates, and export correctness, benchmarked against the manual baseline processes the system replaces. The expected outcome is a functional, tested application, comprehensive technical documentation, and a publication-ready paper.

1. Introduction

Modern conferences concentrate hundreds of attendees, multiple parallel sessions, and time-boxed catering windows into a short operational period. The quality of the attendee experience depends heavily on how efficiently organisers can verify entitlement (who may collect which meal, who may enter which session), observe live operational state (how many attendees have checked in, which sessions are approaching capacity), and communicate changes (venue moves, schedule adjustments) as they happen.

Traditional approaches rely on fragmented, largely manual tools: printed meal tickets, paper attendance registers, and public-address announcements. These methods have three structural weaknesses. First, paper vouchers are trivially duplicated or reused, and manual meal lines create congestion at service points. Second, organisers have no consolidated, real-time view of attendance or consumption, so operational decisions are made on stale or anecdotal information. Third, attendee communication is broadcast-only and unverifiable, with no record of who received which update.

Quick Response (QR) codes provide a mature, standardised mechanism for machine-readable identification [1]. Their capacity, error correction, and low-cost scanning via commodity smartphone cameras have made them the dominant technology for ticketing and verification workflows [2], and QR-based attendance has been proposed as a way to reclaim the lecture time that manual registers consume [3]. Cloud push-messaging services such as Firebase Cloud Messaging (FCM) exist to send messages to mobile devices reliably [6], within the delivery guarantees Section 2 sets out, and modern cross-platform toolkits such as Flutter allow a single codebase to deliver the required scanning, charting, and notification functionality [15].

This project applies these technologies to conference operations as component 4.3b of the ConferenceCheck capstone programme. A sibling component, 4.3a, handles the core attendee check-in capture and offline synchronisation engine. This project does not rebuild those check-in features. Instead, it consumes check-in data through a shared REST API contract and focuses on what happens after check-in: turning raw attendance figures into live analytics for organisers, managing secure meal voucher distribution and redemption, tracking session capacity, and delivering targeted push notifications. These four capabilities are the core deliverables of this project and carry equal weight in its evaluation.

1.1 Problem Statement

Conference organisers lack integrated, trustworthy tools for the operational tasks that follow attendee registration and check-in. Specifically:

- Meal distribution based on printed vouchers or name lists cannot enforce single use. Duplicate collection is difficult to detect at the point of service, and reconciliation after the event is manual and unreliable.

- Attendance and check-in data, even when captured digitally by a check-in system, is not surfaced to organisers in real time. Without a consolidated operational view, decisions about catering quantities, room allocation, and staffing are made on guesswork.
- Session rooms have physical capacity limits, but organisers have no mechanism that tracks per-session attendance as it happens or warns them before rooms overcrowd.
- Communication with attendees is one-directional and unlogged; there is no record of which attendees were reachable or which received a given schedule change.

The consequence is operational inefficiency, measurable fraud exposure in catering budgets, safety risk from overcrowded rooms, and a degraded attendee experience. While the CMS v4 backend manages pre-event registration data, and component 4.3a contributes attendee check-in capture with offline synchronisation, no component currently converts this data into live operational control. This project addresses that gap.

1.2 Project Aim

To design, implement, and evaluate the analytics, meal management, session capacity, notification, and reporting component of the ConferenceCheck Mobile application, providing conference organisers with real-time operational insight, fraud-resistant meal voucher redemption, and auditable attendee communication.

1.3 Objectives

1. To design and implement a real-time analytics dashboard showing check-in progress, check-in rates, and trends for event organisers.
2. To build a QR-code based meal voucher scanning and redemption system that enforces single-use redemption across multiple meal categories.
3. To implement session attendance tracking with capacity management and overcrowding alerts.
4. To build a push notification system for attendee communications and schedule updates, with per-recipient delivery records.
5. To implement reporting and data export (CSV) functionality for event organisers.
6. To evaluate redemption correctness, scan latency, dashboard freshness, notification delivery and export correctness against the Table 3 targets and a manual baseline.

7. To produce comprehensive technical documentation and a publication-ready paper.

1.4 Scope of the Project

In Scope:

- Flutter mobile application modules for the analytics dashboard, meal voucher generation and scanning, session attendance and capacity alerts, notification inbox, and report access.
- Server-side business logic on the CMS v4 Laravel backend for voucher issuance and single-use redemption, session capacity rules, notification dispatch with delivery records, and CSV export endpoints.
- Integration with the 4.3a core check-in module through a shared REST API contract, consuming its check-in data as a read-only input to analytics. This project does not implement check-in scanning, attendee search, or the offline synchronisation engine.
- Role-based access for organiser, scanner, and attendee user types.
- Evaluation on seeded, realistic event datasets including concurrent-scan fraud scenarios.

Out of Scope:

- Attendee check-in capture and QR-based event check-in scanning (delivered by component 4.3a).
- The offline check-in synchronisation engine and its conflict resolution logic (delivered by component 4.3a).
- Walk-in attendee registration and attendee search/lookup workflows (delivered by component 4.3a).
- Payment processing and paid ticketing.
- Integration with external ticketing platforms.
- A full web-based administration panel (organiser features are delivered in the mobile application).

Integration boundary: This component is designed to stand alone for grading purposes. Check-in data is consumed through defined API endpoints. During development and testing, these endpoints

return seeded fixture data so that this component can be built and evaluated without requiring the live 4.3a module. When both components are deployed together, the same endpoints serve real check-in records.

2. Literature Review

The QR Code symbology is standardised in ISO/IEC 18004 [1], which specifies its data capacity and, critically for verification applications, its four selectable error-correction levels, allowing codes to remain machine-readable when partially damaged or poorly displayed on low-brightness screens.

Tiwari [2] surveys the structural properties that drove QR adoption over one-dimensional barcodes in ticketing and access-control contexts: high capacity in small print area, omnidirectional scanning, and decode robustness on commodity camera hardware. A key design implication, adopted in this project, is that the QR payload should be an opaque server-issued token rather than semantically meaningful data, so that possession of the printed code confers no information and validation authority remains entirely server-side.

Masalha and Hirzallah [3] propose a QR-based attendance system for a university setting, motivated by the time manual registers consume. Their paper describes and analyses the design but reports no experiment and no measured comparison against a manual register, so it is not evidence of how much time or error such a system saves. Its analysis does identify the central weakness of naive QR attendance: codes can be photographed and shared, so any anti-fraud property must come from what the server does with a scan, not from the code itself. This finding directly motivates this project's server-side single-use redemption design, in which the database, not the scanning client, is the arbiter of whether a voucher has already been consumed.

For the redemption transaction itself, Helland [11] provides the canonical treatment of idempotence in distributed systems: operations that may be retried (as scans on unreliable venue networks will be) must be designed so that repeated submission cannot produce repeated effect. In this project, that principle is realised by recording redemptions in an append-only table with a database-level uniqueness constraint on the voucher identifier, making double-redemption a constraint violation rather than an application-level race condition.

On the presentation side, Few [4] establishes design principles for operational dashboards: a dashboard must fit on a single screen, prioritise the small number of measures that drive action, and use visual encoding (position, length, slope) rather than tables of numbers for trend perception. Nielsen [5] provides the response-time thresholds that inform this project's latency targets:

feedback within one second preserves a user's flow of thought, which sets the budget for scan-validation round trips at a catering service point where queue throughput matters.

For attendee communication, Google's Firebase Cloud Messaging documentation describes a push architecture in which each app instance has its own registration token [16], and states that a message accepted for delivery may be delayed or not delivered [17]; delivery is therefore best-effort, tokens expire, and devices without Google Play services are unreachable, all of which motivate this project's dual-channel design in which every notification is also persisted to an in-app inbox with per-recipient delivery records. The service interface between the mobile client and backend follows the REST architectural style [7]; endpoint authentication uses bearer tokens as specified in RFC 6750 [8] and implemented by Laravel Sanctum [9], with mobile-specific hardening guided by the OWASP Mobile Application Security Verification Standard [10]. The client architecture applies the separation of presentation, state, and data access descended from the Model-View-Controller pattern [12], realised in Flutter's widget and provider model [15].

Taken together, this body of work indicates that the individual technologies are mature, but that existing systems apply them piecemeal: attendance tools do not manage catering, catering tools do not track session capacity, and none provide the organiser with a consolidated live operational view. Table 1 maps the key findings from prior work to the design decisions of this project.

Source	Key Finding	Mapped Design Decision
ISO/IEC 18004 [1]; Tiwari [2]	QR codes are robust, high-capacity carriers, but payloads are readable by any scanner	QR payload is an opaque unique server-issued token; no entitlement data is stored in the code
Masalha & Hirzallah [3]	QR attendance is faster and more accurate than manual registers, but codes can be copied	All validation is server-side; a copied code cannot be redeemed twice
Helland [11]	Retried operations must be idempotent by design	Append-only redemption table with a unique constraint per voucher; retries are harmless
Few [4]; Nielsen [5]	Operational dashboards need single-screen actionable measures; sub-second feedback preserves flow	Summary-first dashboard design; < 1 s scan validation budget at service points
FCM documentation [6]	Push delivery is best-effort and device-dependent	Dual channel: push plus persisted in-app inbox with per-recipient delivery records
RFC 6750 [8]; OWASP MASVS [10]	Bearer tokens require transport protection and scoped roles	HTTPS-only API, Sanctum token auth, organiser/scanner/attendee role separation

Table 1. Mapping of prior work to the design decisions of this project.

3. Proposed Methodology

The project follows an Agile (Scrum) development methodology [13], organised into two-week sprints with supervisor checkpoints aligned to the departmental progress reviews at weeks 8, 16, and 24. Each functional module (voucher redemption, analytics, sessions, notifications, reporting) is developed as a vertical slice: API endpoint, business rules, database migration, mobile screen, and automated tests are delivered together, so that every sprint ends with a demonstrable increment.

3.1 System Architecture

The system is a client-server architecture with three tiers: the Flutter mobile application, the CMS v4 Laravel backend exposing a REST API [7], and a PostgreSQL database. Figure 1 shows the architecture and the integration boundaries with component 4.3a and with Firebase Cloud Messaging.

ConferenceCheck Mobile — Flutter application (Android)	
Component 4.3a — Core Check-in and Sync Engine <i>Sibling component, developed separately — out of scope for this project</i> • Attendee check-in capture • Offline synchronisation engine	Component 4.3b — Analytics, Meals and Notifications <i>This project</i> • Analytics dashboard • Meal voucher scan and redemption • Session capacity and overcrowding alerts • Notification inbox • Reports and CSV export
<i>Shared REST API contract, Sanctum bearer-token authenticated — 4.3b consumes 4.3a's check-in data as seeded fixtures during independent development</i>	
CMS v4 Laravel REST API + PostgreSQL database (shared backend)	
<i>Push dispatch: Laravel → Firebase Cloud Messaging (external service) → device — component 4.3b notifications only</i>	

Figure 1. System architecture, showing the CMS v4 / PostgreSQL backend shared by both components, and the boundary between component 4.3a (sibling, out of scope) and component 4.3b (this project).

The mobile application follows a layered structure derived from the Model-View-Controller separation [12]: a presentation layer of Flutter widgets; a state layer of Riverpod providers holding screen state and orchestrating asynchronous calls; and a data layer of repository classes wrapping a single authenticated REST client. Platform services (camera access via `mobile_scanner`, push

token registration via firebase_messaging) are isolated behind interfaces so that they can be stubbed in widget tests.

The backend extends the CMS v4 Laravel application. All endpoints require a Sanctum-issued bearer token [8], [9] and return a uniform JSON envelope with success, message, and data fields. Authorisation is role-based: organiser accounts manage events and view analytics, scanner accounts may only submit scan validations, and attendee accounts may only read their own vouchers and notifications. Table 2 summarises the core relations and the integrity constraints on which the anti-fraud properties rest.

Relation	Purpose	Key Integrity Constraint
attendees	Event attendee records (shared with 4.3a)	Unique ticket_code; unique qr_token
meal_categories	Meal windows (e.g. Breakfast, Lunch, VIP Lunch)	Owned by event; time-windowed
meal_vouchers	One voucher per attendee per category	Unique qr_token; unique (attendee, category) pair
meal_redemptions	Immutable record of each redemption	Unique meal_voucher_id enforces single use
event_sessions / session_attendance	Conference sessions and per-session attendance	Unique (session, attendee) prevents duplicate attendance
device_tokens / notification_recipients	FCM registration and per-recipient delivery records	Delivery auditable per user and per notification

Table 2. Core relations and the integrity constraints underpinning system guarantees.

3.2 Meal Voucher Redemption Protocol

Voucher issuance generates exactly one voucher per attendee per meal category, each carrying a cryptographically random, globally unique QR token. The token is opaque: it encodes no attendee or entitlement information, so possession of a screenshot confers nothing beyond what the server chooses to honour [2], [3]. (1) A scanner device decodes the QR token and submits it with its own device identifier to the scan endpoint. (2) The server resolves the token to a voucher, verifying that the voucher belongs to the event being served and that the category's service window is open. (3) Within a database transaction, the server inserts a redemption record; the unique constraint on the voucher identifier guarantees that if two scanners submit the same token concurrently, exactly one insert succeeds and the other receives a definitive 'already redeemed' response naming the earlier redemption time [11]. (4) The redemption record, including the redeeming device identifier and timestamp, is immutable thereafter, providing a complete audit trail for post-event reconciliation.

This design places the single-use guarantee in the database's constraint machinery rather than in application-level checks, eliminating time-of-check to time-of-use races by construction.

3.3 Analytics Dashboard

The dashboard consumes four aggregate endpoints (summary, check-ins over time, meal redemptions by category, session attendance) computed by the backend directly from the operational tables, including check-in data produced by component 4.3a. This project does not implement the check-in capture itself; it reads the check-in records through shared database tables and presents them alongside its own meal and session data. Following Few's single-screen principle [4], the landing view presents total registrations, check-in count and rate, and voucher redemption progress as headline figures, with time-series charts (rendered with `fl_chart`) showing check-in velocity and per-category meal consumption beneath. The client refreshes on a 30-second polling interval, a deliberate trade-off documented in Section 8: polling over the existing REST API avoids introducing a WebSocket layer while keeping data staleness within one operational decision cycle.

3.4 Session Attendance and Capacity Management

Each session carries a room capacity. Attendee entry is recorded by scanning the attendee's QR badge at the session door; the unique (session, attendee) constraint prevents double counting. The backend evaluates occupancy against capacity on every scan and exposes threshold states: normal, warning (occupancy at or above a configurable fraction, default 90%), and full. Threshold transitions trigger an organiser-targeted push notification, converting a safety and comfort problem, room overcrowding, into an alert delivered before the room reaches capacity rather than a discovery made after.

3.5 Push Notification System

Organisers compose notifications targeted at all attendees or at role-scoped groups. The dispatch service resolves the target set into per-recipient delivery records, then delivers via FCM [6] to every registered device token. Because FCM delivery is best-effort [6], every notification is simultaneously persisted and served through an authenticated in-app inbox, so an attendee whose device was offline, or lacks Google Play services, still receives the message on next application

open. The per-recipient records make delivery auditable: organisers can see what fraction of the audience was reachable for any given schedule change.

3.6 Reporting and Data Export

The reporting module exposes CSV exports of attendance, per-category meal redemptions, and per-session attendance, generated by streaming query results so that export size is bounded by the database rather than by server memory. Exported files are the bridge to the organisation's post-event processes (finance reconciliation of catering, sponsor reporting), and their correctness is an explicit evaluation criterion (Section 5).

3.7 Tools and Technologies

- **Mobile:** Flutter 3.x / Dart; Riverpod (state management); fl_chart (charting); qr_flutter (QR rendering); mobile_scanner (camera decoding); firebase_messaging (push).
- **Backend:** Laravel (PHP) REST API on the existing CMS v4 codebase; Laravel Sanctum for token authentication; Pest/PHPUnit for API tests.
- **Database:** PostgreSQL (MySQL and SQLite remain supported through Laravel migrations).
- **Infrastructure and tooling:** Firebase Cloud Messaging; Git for version control; Android Studio / Visual Studio Code.

3.8 Ethical and Legal Considerations

The system processes personal data (names, contact details, attendance behaviour) and therefore observes the data-minimisation and consent requirements of the Data Protection Act No. 3 of 2021 of Zambia [14]. Specifically: attendee records hold only fields required for event operations; QR tokens are opaque and carry no personal data; all client-server traffic is encrypted in transit (HTTPS); API access is authenticated and role-scoped following OWASP MASVS guidance [10]; and evaluation is performed exclusively on synthetically seeded demonstration data, so no real attendee data is processed during development. Proper attribution is given to all libraries, prior work, and external sources.

4. Expected Outcomes

The project will produce the following deliverables, aligned one-to-one with the objectives in Section 1.3:

- A functional Flutter mobile application implementing the analytics dashboard, meal voucher scanning and redemption, session attendance with capacity alerts, notification inbox, and report access.
- Backend API extensions to CMS v4 implementing voucher issuance and single-use redemption, session capacity rules, notification dispatch with delivery records, and CSV export endpoints, with automated test coverage of every business rule.
- A documented REST API contract shared with component 4.3a.
- An evaluation report presenting the results of the acceptance tests defined in Section 5, including concurrent-redemption and baseline-comparison results.
- Comprehensive technical documentation: API reference, database schema documentation, and deployment guide.
- A publication-ready paper covering the design, implementation, and evaluation of the system.

5. Evaluation Methodology

Evaluation proceeds at two levels: correctness of the individual business rules, verified continuously through automated tests, and end-to-end operational performance, verified in a structured simulated-event exercise on seeded data (500 attendees, four meal categories, ten sessions) with scripted fraud and load scenarios.

Automated verification uses Pest/PHPUnit feature tests against every API business rule (voucher uniqueness, redemption single-use, session duplicate rejection, capacity threshold transitions, role-based access denial) and Flutter widget tests for the scanning and dashboard screens. The simulated-event exercise then measures the system against the quantitative targets in Table 3.

Criterion	Target	Measurement Method
Duplicate redemption rejection	100% of duplicate scans rejected, including two devices submitting the same token concurrently	Concurrent scan test: parallel requests with identical tokens; assert exactly one redemption record exists

Scan validation latency	Median < 500 ms, 95th percentile < 1 s on venue-grade Wi-Fi [5]	Timed scan batches from two scanner devices during simulated meal service
Dashboard data freshness	Displayed figures at most 30 s behind database state	Inject check-ins; measure time to dashboard reflection
Capacity alerting	Warning alert delivered to organiser before session occupancy reaches 100%	Scripted session entry until threshold; verify alert receipt order
Notification delivery	95% or more of online devices receive push within 30 s; 100% of recipients see the message via inbox on next open	Delivery-record audit against instrumented test devices
Export correctness	CSV row counts and totals equal database ground truth exactly	Automated comparison of export against direct SQL aggregates

Table 3. Quantitative acceptance criteria and measurement methods.

Baseline comparison. The voucher module is additionally compared against the manual process it replaces: a simulated paper-register meal line staffed by a human checker working from a printed attendee list. Throughput (attendees served per minute) and fraud detection (proportion of scripted duplicate-collection attempts caught) are measured for both conditions. No prior QR attendance study reports such a measured comparison; [3] proposes one design without measuring it, so this comparison is this project's own. The system is expected to dominate the manual baseline on both measures, and the margin is reported explicitly in the evaluation chapter of the final report.

6. Project Timeline

The project runs for 24 weeks, aligned with the departmental progress reviews at weeks 8, 16, and 24. Report chapters are drafted incrementally alongside implementation rather than deferred to the final weeks.

Phase	Weeks	Key Activities
Proposal and Planning	1–2	Topic confirmation, proposal writing, supervisor consultation
Requirements and Design	3–5	Literature review; API contract agreed with 4.3a; database schema design; UI wireframes
Environment and Foundations	6–7	Backend scaffolding on CMS v4; authentication and role model; Flutter project structure; CI test harness
Progress Review 1	8	Demonstration of design artefacts and authenticated skeleton application
Meal Voucher Module	9–11	Voucher issuance, scan endpoint, single-use enforcement, scanner UI; concurrency tests
Analytics and Sessions	12–14	Aggregate endpoints, dashboard screens and charts; session scan, capacity thresholds and alerts
Notifications	15	FCM integration, device token registry, dispatch service, in-app inbox
Progress Review 2 (Midway)	16	End-to-end demonstration; scope adjustment if required
Reporting and Integration	17–18	CSV export endpoints; integration testing with 4.3a check-in data
Evaluation	19–20	Simulated-event exercise; acceptance measurements (Table 3); baseline comparison
Documentation and Paper	21–23	Technical documentation; publication-ready paper; report consolidation (chapters drafted from week 9 onward)
Progress Review 3 / Final Submission	24	Submission of code, report, paper, and presentation

Table 4. Project timeline across the 24-week programme.

7. Risk Analysis

Risk	Likelihood	Mitigation Strategy
QR voucher copying or replay (screenshot fraud)	Medium	Opaque server-issued tokens; server-side single-use enforcement; immutable redemption audit trail with device identifier
Concurrent duplicate redemption at busy service points	Medium	Database-level unique constraint inside a transaction; explicit concurrent-scan acceptance test
Unreliable venue network connectivity	High	Sub-second, retry-safe (idempotent) scan endpoint; client retry with exponential backoff; graceful degradation of dashboard to last-known data

Variable or failed FCM push delivery	Medium	Persisted in-app inbox as guaranteed second channel; per-recipient delivery records to quantify reach
Dependency on 4.3a integration	Medium	API contract agreed and frozen in week 5; seeded check-in fixtures allow development and testing without the live 4.3a module
Schedule overrun	Medium	Two-week sprints with vertical slices; modules ordered by objective priority so a slip degrades scope, not core guarantees
Device fragmentation	Low	Target Android 8.0+; test matrix of two physical devices plus emulator profiles

Table 5. Risk register with likelihood and mitigation strategies.

8. Limitations

Several limitations of the proposed design are acknowledged.

The analytics dashboard is near-real-time rather than real-time: a 30-second polling interval bounds data staleness but does not deliver push-driven updates. This is a deliberate trade-off to avoid introducing a WebSocket infrastructure alongside the existing REST API within the project timeline; the polling interval is well within the decision cycle of on-site operations, but the limitation is documented and a streaming upgrade path is identified as future work.

Voucher redemption requires connectivity at the point of scanning. Offline check-in synchronisation is the remit of component 4.3a and is intentionally out of scope here; a fully offline redemption mode would require conflict resolution for the single-use guarantee that cannot be provided without connectivity, so service points are assumed to have at least intermittent network access.

Push notification delivery depends on Firebase Cloud Messaging and therefore on Google Play services; devices without them are reachable only through the in-app inbox channel. Reported delivery rates in the evaluation are conditioned on this constraint.

Evaluation is conducted on a structured simulated event with seeded data and scripted scenarios rather than a live conference deployment. This provides controlled, reproducible measurements, including fraud scenarios that could not ethically be staged at a real event, but leaves live-deployment validation, with its full range of human and environmental variability, to future work.

Finally, primary development and testing target Android devices; iOS compatibility is expected from Flutter's cross-platform model but is not systematically verified within this project.

9. AI-Assisted Development Statement

In accordance with the supervision framework for the 2025/2026 academic year, this section discloses the planned use of AI-assisted tools during the development of this project.

9.1 AI Tools to be Used

The following AI tools are planned for use: (1) GitHub Copilot, for in-editor code suggestions during implementation of the Flutter screens and Laravel endpoints; (2) Claude (Anthropic), for reviewing draft documentation, assisting with the structure of written sections, and explaining framework APIs; and (3) ChatGPT (OpenAI), as a secondary tool for exploratory discussion of design decisions and debugging assistance.

9.2 Intended Tasks and Scope of Assistance

AI assistance will be limited to: (a) boilerplate generation for data models, REST client wrappers, and test scaffolding, where the design remains the student's own; (b) proofreading and improving clarity of written documentation without altering analytical content or conclusions; and (c) clarifying questions about library APIs and language syntax. AI tools will not be used to design the voucher redemption protocol, the capacity alerting rules, the evaluation methodology, or the interpretation of evaluation results, as these constitute the academic contribution of the project.

9.3 Validation of AI-Generated Code

All AI-generated or AI-assisted code will be reviewed line-by-line before integration. Validation will be performed through: (i) automated testing (Pest/PHPUnit for the API, flutter test for the client), with test cases designed independently of any AI suggestion; (ii) manual trace-through of business-rule logic against known input-output pairs; and (iii) cross-referencing against official library documentation. Any code segment that cannot be fully understood and explained by the student will not be incorporated into the project deliverables.

9.4 Documentation of AI Usage

AI usage will be documented throughout the project in a dedicated AI Usage Log maintained as an appendix to the final report, recording for each instance the tool used, the date, the task posed,

a summary of the output, and the extent to which the output was used, modified, or rejected. The log will be submitted alongside the code repository and final report.

10. Remaining Work

This account is dated from the resubmission on 10 October 2026, not from the cover date. It answers the comments on submissions 116 (final report), 117 (validation document) and 118 (this proposal). The milestone tags in the repository were created in September and mark representative commits, so each completed item below links its own commit and evidence; docs/remaining_work.md in the repository gives the commit and test for every item.

Table 6. Work completed since the comments, with its evidence.

Date	Item	Evidence
07/10/26	Authorisation: attendees open only their own vouchers, reports are organiser only, notifications are readable only by their recipients, and four related fixes	Negative tests expecting 403 in tests/Feature/ConferenceApiTest.php; commits a6596e8 to 85b529c
09/10/26	Redemption record protected from edits and cascading deletes; 409 instead of 500 when the unique constraint refuses a scan; locks on session and check-in scans; chunked exports; Firebase demo mode never recorded as delivery	Feature tests; commits da5a91d to ab206f4
09-10/10/26	Offline replay keeps recoverable failures and no longer overwrites a scan queued during replay	mobile/test/scan_replay_test.dart and scan_replay_restart_test.dart; commits 3387288, 6ebc456, 33d3c89
10/10/26	Concurrent redemption on PostgreSQL behind five server processes, with the row lock and unique constraint each removed as controls	tools/concurrency-results.json; commit e9aab32
10/10/26	All four CSV exports checked against direct SQL on rows and totals; evaluation rerun	tools/evaluation-results.json; commit 907f237
10/10/26	Dashboard freshness measured by injecting check-ins and timing their appearance	tools/freshness-results.json; commit 094e5e1
10/10/26	Report and validation document corrected and rebuilt, with a per-criterion status table	docs/deliverables/; commits 19c14e1 to b4617f8

Table 7. Work still to be done.

Item	Due
------	-----

Upload this proposal (v4) to the Project Proposal slot	10/10/26
Upload the corrected final report and validation document, with the final commit tagged final-resubmission	10/10/26
Push delivery measured across a fleet of devices, a timed capacity alert, and the manual baseline: not possible within the project and stated as limitations in the final report	Not scheduled

11. References

- [1] ISO/IEC 18004:2015, Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification. Geneva, Switzerland: International Organization for Standardization, 2015.
- [2] S. Tiwari, "An introduction to QR code technology," in Proc. 2016 Int. Conf. on Information Technology (ICIT), Bhubaneswar, India, 2016, pp. 39–44.
- [3] F. Masalha and N. Hirzallah, "A students attendance system using QR code," International Journal of Advanced Computer Science and Applications, vol. 5, no. 3, pp. 75–79, 2014.
- [4] S. Few, Information Dashboard Design: The Effective Visual Communication of Data. Sebastopol, CA, USA: O'Reilly Media, 2006.
- [5] J. Nielsen, Usability Engineering. San Francisco, CA, USA: Morgan Kaufmann, 1993.
- [6] Google, "Firebase Cloud Messaging," Firebase Documentation, 2025. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>
- [7] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.
- [8] M. Jones and D. Hardt, "The OAuth 2.0 Authorization Framework: Bearer Token Usage," IETF RFC 6750, Oct. 2012.
- [9] Laravel, "Laravel Sanctum," Laravel Documentation, 2025. [Online]. Available: <https://laravel.com/docs/sanctum>
- [10] OWASP Foundation, OWASP Mobile Application Security Verification Standard (MASVS), v2.0, 2023. [Online]. Available: <https://mas.owasp.org>
- [11] P. Helland, "Idempotence is not a medical condition," Communications of the ACM, vol. 55, no. 5, pp. 56–65, 2012.

- [12] G. E. Krasner and S. T. Pope, "A cookbook for using the Model-View-Controller user interface paradigm in Smalltalk-80," *Journal of Object-Oriented Programming*, vol. 1, no. 3, pp. 26–49, 1988.
- [13] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020. [Online]. Available: <https://scrumguides.org>
- [14] Republic of Zambia, *Data Protection Act No. 3 of 2021*. Lusaka: Government Printer, 2021.
- [15] Google, "Flutter documentation," 2025. [Online]. Available: <https://docs.flutter.dev>
- [16] Google, "FCM architectural overview," *Firebase Documentation*, 2025. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging/fcm-architecture>
- [17] Google, "Understanding message delivery," *Firebase Documentation*, 2025. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging/understand-delivery>